

ADR 0137 — An untrusted checkout inherits an allowlist, and the phase that runs attacker code gives up what it does not need

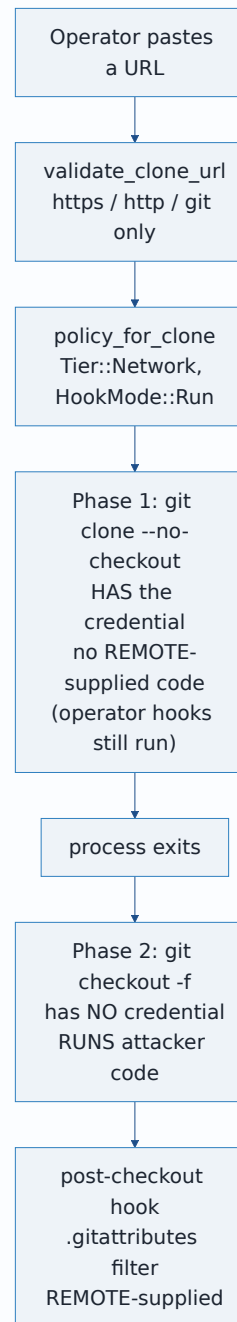
- **Status:** Accepted — implemented, mutation-proved two ways per issue, all four failing differently
- **Date:** 2026-09-07
- **Issues:** #702, #704 — one root cause seen from two sides
- **Extends:** [ADR 0128](#) (the credential boundary this widens from three names to a built environment)
- **Supersedes in part:** [ADR 0033](#) — its safety argument for granting the SSH agent socket and `known_hosts` to `policy_for_clone`. ADR 0033 stands unchanged for `policy_for`.
- **Related:** [ADR 0028](#) (a port grant is not an egress policy — read before believing the port half buys more than it does), [ADR 0122](#), [ADR 0123](#)

Context

`POST /api/clone` is loopback-only and needs a live session and a CSRF token, so nothing here is an unauthenticated bypass. The trigger is the *expected* product action: an authenticated operator clones a repository they do not control.

ADR 0128 split that operation in two. A credentialed `git clone --no-checkout` fetches objects and exits; a second, credentialless process runs `git checkout -f`, which is where attacker-selected

`post-checkout` hooks and `.gitattributes`-selected content filters execute. Running them is deliberate product behaviour, not an oversight — ADR 0128 rejected disabling them by name.



The boundary between those phases was

`SandboxedCommand::without_credential_env`, and it removed exactly three names: `GIT_VISTA_CREDENTIAL_TOKEN`, `GIT_VISTA_GITHUB_TOKEN`, `GH_TOKEN`. Everything else in the server's ambient environment was inherited verbatim.

#704 — the list was only ever as complete as the last person to think about it

Any credential the operator exported before starting the server reached attacker-selected code: `AWS_SECRET_ACCESS_KEY`, `NPM_TOKEN`, a CI runner's injected secret. Reproduced dynamically on `a6acf693` with a synthetic canary; no real credential was read. The defect is not any particular missing name — it is that a *removal list* is the wrong shape for the question "what may this process see."

#702 — the specific one, and it was already contradicted in writing

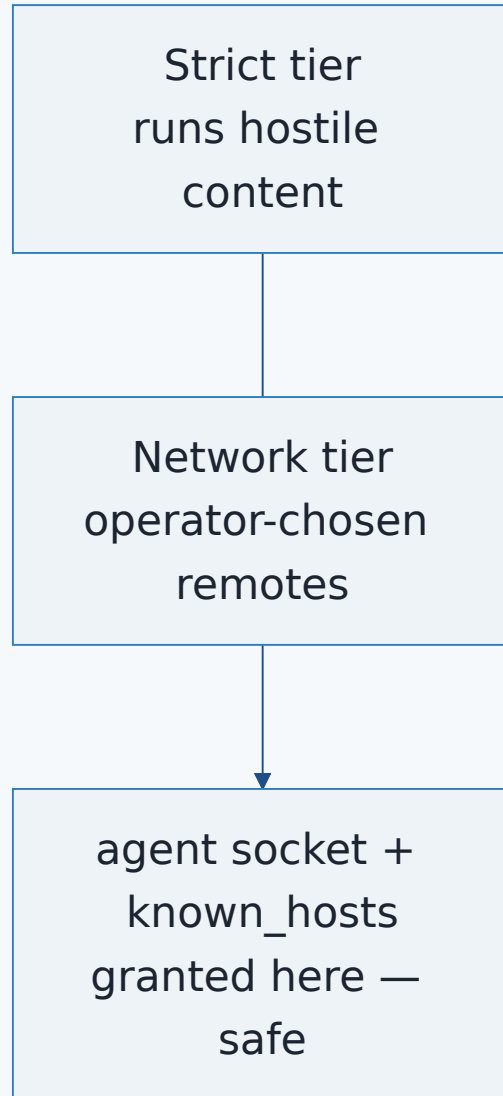
`policy_for_clone` is an independent `Policy` constructor. It copied #188's SSH carve-out into every clone: `$SSH_AUTH_SOCK` into `rw_trees`, `~/.ssh/known_hosts` into `ro_carveouts`, and the full `DEFAULT_GIT_PORTS` including 22. So a fetched `post-checkout` hook inherited the operator's live agent socket. An agent will not surrender private-key bytes, but it will happily *sign* for a process that asks — and a Network-tier port grant is port-only, never host-bound (ADR 0028), so the same hook could reach port 22 anywhere. Proved dynamically on `a6acf693` with a throwaway key and a throwaway agent.

ADR 0033 permitted those grants on an argument it stated plainly:

Neither grant reaches the Strict tier, which is where hostile repository content actually runs; both apply only to push/fetch/clone/`ls-remote`, which are user-initiated against a remote the user chose.

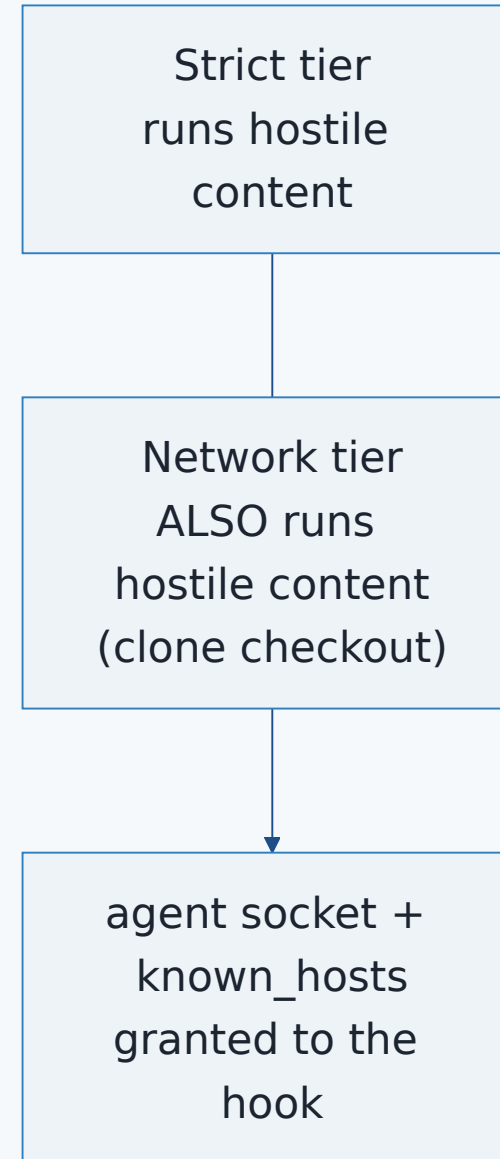
That was true in July. ADR 0128 made it false in September: hostile repository content now runs in the Network tier, by design, in this exact policy. Nothing about #680's change was wrong — but a safety argument written against the old arrangement was left standing under the new one.

ADR 0033's premise (2026-07-31)



#680 splits clone;
the premise stops holding

After ADR 0128 (2026-09-06)



A premise that was false, and how it survived two readers

The first version of this change removed both #188 grants and port 22 from `policy_for_clone` outright, on this argument:

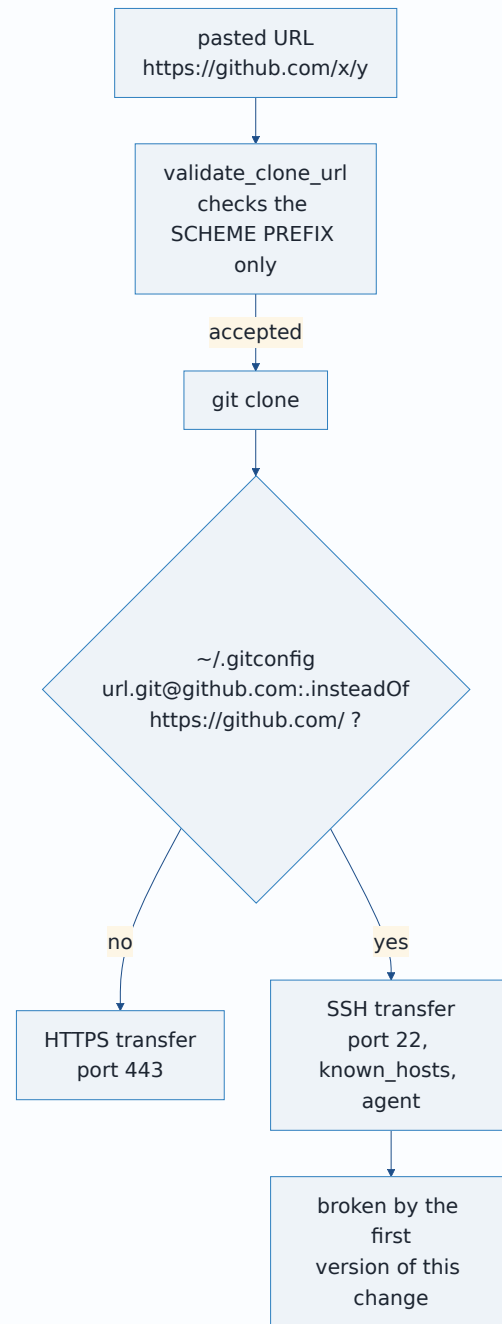
`validate_clone_url` accepts `https://`, `http://` and `git://` and rejects everything else. This route **cannot perform an SSH clone**. A capability no code path on this route can legitimately reach is not a trade-off to weigh.

That is false, and the change was a regression.

`validate_clone_url` is a scheme-prefix check with no port or transport constraint. Two paths reach SSH without ever failing it:

- `https://host:22/repo.git` is accepted — the validator never looks at the port;
- `url.<base>.insteadOf` in the operator's own `~/.gitconfig` rewrites an accepted `https://` URL into a real SSH clone — using the very global config this ADR's own allowlist deliberately preserves, two sections below.

So operators with `insteadOf` configured had **working clones broken** by the removal: no `known_hosts` carve-out means host-key verification fails, and no port 22 means the connection is denied outright. Caught in review by codex-daybreak on #720, traced to source, before it merged.



The transferable lesson is that the failure was a premise, not an implementation. "A capability no code path can reach" was checked against the URL *scheme* — the one path that names the capability — and never against ports, config rewriting, or `insteadOf`. It began in the lane handoff, survived the lane's own verification pass, and was refuted only by a fresh reader who went to `dto.rs` instead of accepting the sentence. A capability argument has to be checked against **every** path that can reach the capability, not the one that names it.

The correct statement is narrower: **whatever the process that runs remote-supplied code needs, it may not safely have.** That is a claim about a *phase* rather than a route, which is why the fix below splits the policy instead of narrowing a shared one.

Note the wording carefully. An earlier draft said the phase "does not need these", and that is also false — SSH-backed Git LFS needs exactly these, at checkout time. The withholding is not justified by the phase having no use for the capability; it is justified by the phase being the one that runs attacker-selected code, which makes the capability unsafe to grant *whether or not something legitimate wants it*. The cost of that is real and is accounted for under "What this does not close".

Decision

1. The untrusted checkout's environment is built, not filtered

`spawn::UNTRUSTED_CHECKOUT_ENV_ALLOWLIST` is the complete set of names the checkout child may inherit.

`with_untrusted_checkout_env` calls `env_clear()` and then copies across only those names that the parent actually has.

Before — denylist (ADR 0128)

inherit
EVERYTHING

remove 3 known
names

child sees
everything else

#704

After — allowlist (#704)

env_clear()
start from
nothing

copy 7 named,
justified variables

child sees
only those

Name	Why it survives the cut
PATH	gv-sandbox reaches git through <code>Command::new("git").exec()</code> , a <code>PATH</code> lookup. Without it there is no checkout at all, not a reduced one. Hooks and filters need it too.
HOME	Git resolves <code>~/.gitconfig</code> through it. ADR 0128 is explicit that the split changes <i>when</i> checkout happens, not whether hooks and filters run — dropping <code>HOME</code> would silently stop operator-configured ones. The secrets under <code>\$HOME</code> stay withheld by <code>secret_excludes</code> regardless.
XDG_CONFIG_HOME	The other root git consults for global config. Omitting it would make config resolution differ between the two phases for exactly the operators who use it.
LANG, LC_ALL, LC_CTYPE, LC_MESSAGES	Locale. They select message text and character handling; they name no resource and grant no access.

Three exclusions are decisions rather than omissions, and are recorded as such in the constant's own doc comment:

- `SSH_AUTH_SOCK` — #702's *mechanism*: the variable is how the operator's agent becomes reachable in practice, so withholding it is the highest-value single line here. It is **not** a fix for #702, which stays open — the capability survives without the locator. See "What this does not close".

- `GIT_CONFIG_COUNT` / `GIT_CONFIG_KEY_n` / `GIT_CONFIG_VALUE_n`, `GIT_CONFIG_GLOBAL`, `GIT_CONFIG_SYSTEM` — these would buy the same config parity `HOME` buys, but their payload is an arbitrary string and `credential.helper=!echo password=...` is a legal value. A config channel that can carry a secret is not forwarded into attacker-selected code. An operator who configures the server this way loses that configuration at checkout; that is the trade, taken deliberately.
- `TMPDIR`, `TERM`, `TZ` — `git checkout -f` needs none of them, and `/tmp` is in no grant this policy gives out in any case.

Adding a name to that list is a security decision and must arrive with the sentence saying what breaks without it, in the same edit.

2. The guarantee lives in a type, not in a caller's memory

`network_command_without_credential` now returns `UntrustedCheckoutCommand` rather than a bare `SandboxedCommand`. The only constructor applies the allowlist unconditionally; the only completion path redacts URL userinfo. There is no `env`, no `spawn`, and no way to recover the inner command.

This is the same move ADR 0128 made for `CredentialedCommand`, for the same reason, and it removes a second "correct if the caller remembers": the handler previously had to write `.map(redact_output)` on each of its two completion paths. `SandboxedCommand::without_credential_env` is **deleted**, not deprecated — while a "remove these names" builder exists on that type, the next credential-adjacent call site can reach for it and rebuild #704 one variable later.

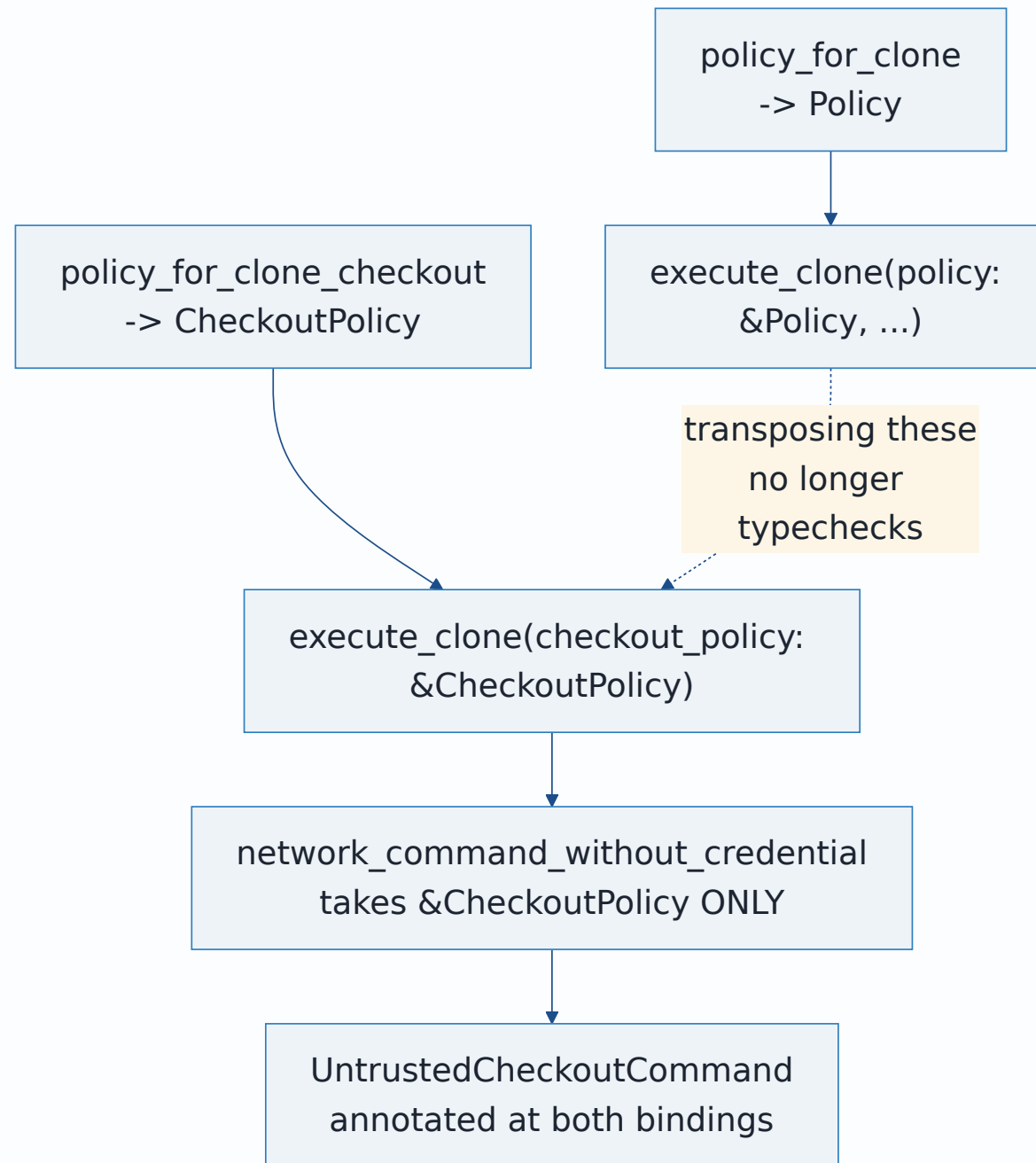
2a. The split is enforced by types, because a scan was not enough

The first version of §3 gave `execute_clone` two `&Policy` arguments and pinned the wiring with a test that read the function's source. `codex-daybreak` defeated it in two lines:

- **transposing the two arguments at the call site** hands the checkout the transfer's SSH grants — the exact outcome the split exists to prevent — and the scan, reading only `execute_clone`'s own body, stayed green;
- **aliasing the raw builder**, use `network_exec::network_command` as `network_command_without_credential`, satisfied every count while returning a command carrying the server's whole inherited environment.

This is #704's own shape a third time: a check that can be satisfied without the property holding. So both are compile errors now.

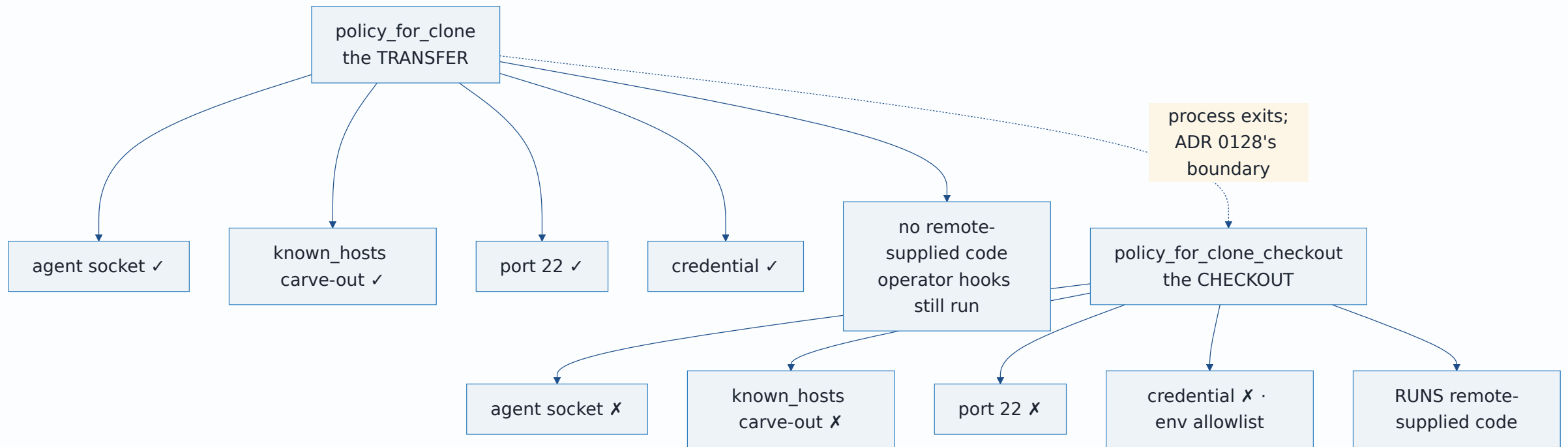
`sandbox::CheckoutPolicy` is a newtype over `Policy` whose field is private to the `sandbox` module, constructible only by `policy_for_clone_checkout`. `network_command_without_credential` accepts nothing else, so an untrusted-checkout launcher cannot be built from a transfer policy at all, and the transposition does not typecheck. `execute_clone` then annotates both command bindings as `UntrustedCheckoutCommand`, which no alias of `network_command` can produce.



The source scan survives, demoted to what it still buys — noticing if one of the two spawns is deleted outright — with its doc saying plainly that it is not the proof. A scan that looks like a boundary invites someone to trust it as one.

3. The two phases get two policies, and the grants follow the split

ADR 0128 split clone into two processes with a security boundary between them — and then handed both of them the same `Policy`. The credential stopped crossing that boundary; every capability kept crossing it. This is that split finally reaching the grants.



`policy_for_clone` keeps all of #188, because the transfer genuinely may need it and the section above shows why. `policy_for_clone_checkout` is that policy minus three things: the `$SSH_AUTH_SOCK` `rw_trees` grant, the `~/.ssh/known_hosts` carve-out, and port 22 (`CLONE_CHECKOUT_PORTS`).

Network access, `HookMode::Run` and filter execution are **unchanged in both**. ADR 0128 kept them deliberately and this narrows what the second process is handed, never what it may do.

"The transfer runs no attacker code" is narrower than it sounds, and the distinction the split rests on is authorship, not data. `--no-checkout` stops remote-supplied hooks and filters, but the transfer is not callback-free: an operator-configured `reference-transaction` hook runs on **remote-controlled refs**, and configured transport and credential helpers execute — all with the transfer's full #188 grants live. Git documents that hook for any ref-updating command. The split still holds, because those programs are chosen by the operator rather than supplied by the remote, which is exactly the line being drawn. But the transfer runs *operator code over attacker-influenced data*, and a later reader should not take the shorter phrase for more than that. Raised by codex-daybreak; neither the lane nor the coordinator had asked about it.

Port 22's justification is now about the phase, not the URL: `git checkout -f` is local, every object is already on disk, and the only legitimate outbound traffic is a content filter's own (`git-lfs` smudge over HTTPS). Whatever the remote's transport was, nothing this process does needs SSH.

Dropping the carve-out also removes a real failure mode: a `stow/chezmoi` user whose `~/.ssh/known_hosts` is a symlink had checkout refused outright by `add_carveout_rule`'s deliberate

`Symlinked` guard (ADR 0033 §1a).

Honest ordering of what closes what. ADR 0033 §3 measured — and this change re-read rather than assumed — that the `rw_trees` agent-socket grant is **inert on this kernel**: Landlock ABI 8 does not mediate pathname `AF_UNIX` sockets, and `ssh_remote.rs`'s own live test still proves it. So §1's allowlist is the load-bearing half of #702. §3 contributes three other things: the `argv` stops advertising a grant the process must not have (ADR 0033's D5 Option B auditability), the grant cannot silently become load-bearing again under a future ABI, and port 22 is no longer reachable.

Rejected alternatives

Add `SSH_AUTH_SOCK` to the removal list

The literal minimum fix for #702, and it is #704 restated. A fourth name makes the fifth omission the next issue. Rejected on shape, not on cost — this is ADR 0123's rule applied to an environment instead of an `argv`.

Run the checkout at the Strict tier instead

Genuinely stronger: Strict denies `AF_UNIX` at the `seccomp` layer outright, so the residual below would be gone. Rejected because Strict has no network, and ADR 0128 kept network access for the checkout deliberately — a `git-lfs` smudge filter is a legitimate checkout-time network consumer.

An earlier draft of this paragraph called that residual "already unreachable", which contradicted this ADR's own limits section three pages later and was the weaker of the two statements. It is struck. The residual is **reachable** — see "What this does not

close" — and the reason not to move checkout to Strict is that it costs a working product feature, not that there is nothing left to gain.

723 buys the same ground without that cost.

Keep `known_hosts` because it is only public key material

True as far as it goes — ADR 0033 is right that `known_hosts` reveals which hosts the operator connects to rather than a secret that grants access.

The first version of this ADR rejected keeping it with "this route cannot verify an SSH host key because it cannot speak SSH" — exactly the false premise recorded above, and the reason the *transfer* keeps the carve-out today. The surviving reason applies to the checkout only: that process verifies no host keys because it opens no connections to hosts, so for it the carve-out is a `~/.ssh` exclude bypass left armed with no caller. The list of hosts an operator connects to is also not nothing to hand a hostile hook.

Leave the environment inherited and sandbox harder instead

Rejected because the filesystem sandbox never saw the leak. An environment variable is not a path; Landlock has no opinion about it. The only layer that can answer "what may this process read out of its own environment" is the layer that builds the environment.

Consequences and limits

- **A legitimate operator hook loses variables it may have relied on.** A hook selected by the operator's own `core.hooksPath` now runs with seven variables rather than the server's whole environment. This is a real behaviour change and the intended one: the boundary cannot distinguish the operator's hook from the remote's, because at checkout time the remote chose which file runs.
- **#702 stays open — see "What this does not close" below.**
- **A proxied or custom-CA `git-lfs` smudge filter fails at checkout.** `HTTPS_PROXY`, `HTTP_PROXY`, `NO_PROXY`, `SSL_CERT_FILE` and `SSL_CERT_DIR` are **not** allowlisted, and networked filters commonly need them — which sits awkwardly beside LFS being the stated reason checkout keeps network at all. The trade is deliberate: a proxy URL carries credentials in its userinfo (`https://user:pass@proxy:3128`), which is precisely the class of value this boundary exists to withhold from attacker-selected code, and it is the commonest secret-bearing variable in CI.
What would reopen this: a way to pass proxy configuration without its userinfo — a sanitised value the server writes into the child, or `http.proxy` as a `-c` flag the server authors — at which point the names can be dropped for good rather than added.
- **Dropping port 22 confines nothing to the operator's own remote.** ADR 0028's limitation is unchanged: a port grant carries no address. It removes one destination service class, and that is all it should ever be described as.
- **`sandbox::test_env` is now the crate's one test-side writer of process environment.** `argv.rs`'s file-scoped `SSH_AUTH_SOCK_LOCK` justified itself on "nothing outside this file touches this key", which #704's proof made false — the fix reads the whole environ, and the clone proof sets the same key.

A lock whose scope is narrower than its key's readership is a comment, not a lock. Callers hold it for *composition* only, never across an `await`.

- **A future `ro_carveouts` caller inherits an empty precedent here.** ADR 0033 said nothing about its shape recommends a second caller; there is now one fewer, not one more.

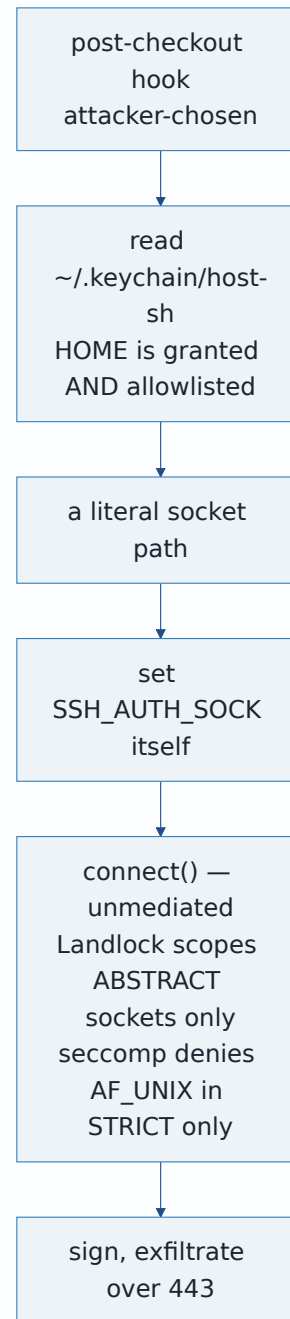
What this does not close

#702 is not closed by this ADR. The allowlist withholds the agent socket's *locator*; it does not deny the *capability*.

`seccomp_filter::af_unix_rule` denies `AF_UNIX` in the **Strict** tier only, and Landlock does not mediate pathname `AF_UNIX` `connect()` at all (ADR 0033 §3). So a hook that recovers a socket path can set `SSH_AUTH_SOCK` itself and connect. Recovery does not need enumeration, which is where this ADR's first draft went wrong:

- `$HOME` is both read-granted and allowlisted, and `~/.keychain/<host>-sh` — an ordinary, common tool — contains the literal line `SSH_AUTH_SOCK=/tmp/ssh-XXXX/agent.NNN; export SSH_AUTH_SOCK; ;`
- shell rc files and desktop-agent conventions give further derivable paths;
- signatures then leave over port **443**, which is allowed and must stay allowed.

Dropping port 22 does not prevent this: the agent protocol is spoken over the local socket, not over TCP 22.



An SSH-resolving Git LFS endpoint fails the clone outright, deliberately

Scope: the axis is the resolved LFS endpoint, not the git transport. Git LFS picks its endpoint from `lfs.url`, then `remote.<name>.lfsurl`, then the remote URL. Affected configurations are the ones whose *resolved endpoint* invokes SSH — hybrid `git-lfs-authenticate` over SSH, or the pure-SSH transfer adapter. Both directions of the obvious guess are wrong: an SSH git remote with an explicit HTTPS `lfs.url` is **unaffected**, and an **HTTPS** git remote whose config selects an SSH LFS endpoint **is** affected. An earlier draft said "SSH-backed clones", which was overbroad in one direction and blind in the other.

Severity: the clone fails and its destination is removed. An earlier draft said such a clone would retrieve pointers and not contents. That is wrong and understates it. A failing smudge filter makes `git checkout -f` exit nonzero; `execute_clone` returns `CloneExecutionError::GitFailed`; `run_guarded`'s `DestGuard` is still armed on that path and deletes the destination directory. The operator gets a failed clone and no repository, not a degraded one. Traced by codex-daybreak through `handlers/clone.rs`.

Accepted rather than fixed, because the fix and the vulnerability are the same thing. Reaching SSH at smudge time means the agent socket, host keys and port 22 present in the process that runs attacker-selected filters — precisely the exposure #702 exists to remove. The sandbox cannot distinguish `git-lfs`'s own `ssh` from a fetched smudge filter's: same process tree, same policy, no signal to separate them. Restoring the grants would undo the change rather than complete it.

What would reopen it — and what would not. An earlier draft proposed that the server run `git-lfs-authenticate` itself and hand the filter the resulting token, under the slogan "pass the

result of using a credential, never the credential". **That slogan is wrong here and the proposal does not have the property it claims.** What `git-lfs-authenticate` returns *is* a bearer credential — an `Authorization` header good for the repository's LFS objects — so handing it to the filter puts a credential inside the untrusted process after all. It also does nothing for a pure-SSH transfer adapter, which produces no HTTPS token to pass. The coordinator endorsed that framing and I wrote it; codex-daybreak refuted it. It is recorded here rather than quietly replaced, because "we already solved that" is how a hole gets inherited.

Two conditions would genuinely reopen it, and they differ in kind:

1. **Keep every credential out of the untrusted process.** The server resolves LFS objects itself — prefetching them between the transfer and the checkout, or proxying the filter's requests through a local endpoint that holds the credential — so the child needs no token and no socket. This is the only option that preserves the property #702 is about, and it also covers pure-SSH, since the server can speak whatever transport the endpoint wants.
2. **Accept a bounded exposure, explicitly.** Hand the filter a repository-scoped, short-lived download credential and say plainly that a hostile filter can use it for its lifetime and its scope. That is a smaller exposure than the agent socket by a wide margin — one repository's objects versus signing for every host the operator can reach — but it is not zero, and it must be argued as a trade rather than described as a fix.

Option 1 is the better answer and the larger piece of work. Neither is in scope here; both are named so the next person starts from an accurate statement of what is left.

Closing it needs a checkout-specific seccomp mode denying pathname `AF_UNIX` while keeping TCP for LFS — `bin/gv-sandbox/seccomp_filter.rs`, tracked as **#723**. It does *not* require

moving checkout to the Strict tier.

What this ADR buys against that path is real but partial: the common case, where the variable is simply inherited, is closed; the argv no longer advertises a grant the process must not have; and the grant cannot become load-bearing again under a future Landlock ABI.

Proof

Both acceptance criteria are dynamic and both run against compiled, host-native code — no `cfg`-gated arm, so nothing here reports green over its own absence.

- `handlers::clone::clone_checkout_runs_the_hook_with_only_an_allowlisted_environment` is #680's canary widened. It builds a source repository with a tracked executable `hooks/post-checkout`, performs the credentialed `--no-checkout` transfer, confirms no content and no marker, selects the hook with `core.hooksPath=hooks`, and runs the production credentialless checkout with a canary variable and a fake `SSH_AUTH_SOCK` genuinely present in the server process. The hook reports seven fields: the three ADR 0128 names, `SSH_AUTH_SOCK`, the canary — all five `unset` — then `PATH` and `HOME`, which must be **present**. An empty environment would satisfy the first five legs while producing a checkout that cannot run, which is why the last two exist. The premise is asserted, not assumed: the canary and the socket are checked present in this process at the moment the command is composed.
- `sandbox::spawn::tests::an_untrusted_checkout_environment_is_built_by_allowlist_not_by_removal` makes the same claim where the decision is made, on a synthetic environment, with `AWS_SECRET_ACCESS_KEY` standing in for "a credential nobody enumerated". A test that checks only the three ADR 0128

names passes identically under a denylist and an allowlist and therefore cannot fail on #704 at all; an unknown name is what separates the two mechanisms.

- `sandbox::argv::only_the_clone_checkout_phase_gives_up_the_18_8_grants` asserts a *difference between three constructors* in one critical section: the same socket path present in `policy_for`'s Network policy **and** in clone's transfer policy, absent only from the checkout's. Without the first two legs the test would pass just as well if `ssh_agent_socket_grant` were broken outright, or if the transfer had been left narrowed and SSH clones still refused — which is the regression this PR had to undo.

`failure-atlas mutation_check`, run key `gv-702-704-untrusted-checkout-allowlist`, HEAD `740acc1b`, clean tree. Every baseline green at 56 passed; every mutation caught.

Run	Claim	Mutation	Kind	What went red, and why it is a <i>different</i> failure
422	#704 env	drop the <code>.filter(...)</code> from <code>untrusted_checkout_env</code>	remove	Three tests. The real spawned hook reported <code>unset\ unset\ /tmp/gv702-not-a-real-agent.sock\ a-credential-nobody-enumerated\ PATH-present\ HOME-present</code> — socket and canary arriving verbatim in attacker-selected code. The defect itself, reproduced.

Run	Claim	Mutation	Kind	What went red, and why it is a <i>different</i> failure
423	#702 env	add "SSH_AUTH_SOCK" to the allowlist	weaken	Two tests; the hook saw the socket back with the canary still withheld. The policy/argv test stayed green — same issue, opposite layer from 428.

Run	Claim	Mutation	Kind	What went red, and why it is a <i>different</i> failure
421	#704 list	delete "XDG_CONFIG_HOME" from the allowlist	weaken, other direction	Only the exact-constant assertion. The arm this ADR's first version could not make at all: its test supplied three allowed names, so removing one changed real behaviour and stayed green.

Run	Claim	Mutation	Kind	What went red, and why it is a <i>different</i> failure
428	#702 policy	policy_for_clone_checkout keeps the transfer's ro_carveouts and net_ports	remove	Only the three-way policy comparison, on ro_carveouts. A pure policy-shape failure; nothing spawned, no environment read.

And one arm the compiler answers instead of a test.

Transposing `execute_clone`'s two policy arguments at the call site — codex-daybreak's demonstrated defeat of the earlier source-level guard — was submitted as run **430** and came back `build_failed`:

```
error[E0308]: arguments to this function are incorrect
845 |             &checkout_policy,
    |             ----- expected `&sandbox::Policy`,
846 |             &policy,
    |             ----- expected `&CheckoutPolicy`, found `&s
```

`failure-atlas` correctly **refuses a verdict** there: a mutation that does not compile says nothing about any assertion, and the tool declines rather than reporting a false `caught`. That refusal is right and this table does not launder it into one. But for *this* claim the

build failure is the stronger evidence, because it is the property itself: the mistake is not caught, it is **unrepresentable**. A test proving the transposition is detected would be a weaker statement than rustc proving it cannot be written.

Read 428 and 422/423 as halves that share no code: a `Policy` built in memory and never run, versus a real `post-checkout` hook spawned under the real shim reporting what it inherited. Either alone licenses a wrong conclusion — 428 alone would say the policy edit is the fix, which ADR 0033 §3's measurement contradicts; 422 alone would leave the `argv` still advertising a grant the process must not have.

Runs 419 and 420 belonged to the earlier, defeatable form of §2a and are superseded: the properties they tested are now carried by types, so there is nothing left for a mutation to survive.

Run 421 is the arm this ADR earned in review. Every other mutation *adds* something dangerous, and a test that pins only the effect on a fixed sample catches those while missing the removal of a name that changes behaviour. Both directions are now pinned by asserting the constant in full.

Two cautions recorded for the next lane. `cargo test -p <crate> -- bins` does **not** build `target/debug/gv-sandbox` (`dev` says so at its own line 234), so every test that builds a real production `Policy` fails on a missing shim in a fresh worktree; and `mutation_check` — which compiles in its own clone but **runs from the source worktree's target dir** — reports that as `baseline_failed` rather than a readable failure. Run `cargo build -p git-vista-server -- bins` once in the worktree first; it cannot go inside `test_commands`, which accepts only test runners.

Where this is implemented

- `crates/git-vista-server/src/sandbox/spawn.rs` — `UNTRUSTED_CHECKOUT_ENV_ALLOWLIST`, `untrusted_checkout_env`, `with_untrusted_checkout_env`; `without_credential_env` deleted.
- `crates/git-vista-server/src/sandbox/network_exec.rs` — `UntrustedCheckoutCommand`; `network_command_without_credential` retyped.
- `crates/git-vista-server/src/sandbox/mod.rs` — `CLONE_CHECKOUT_PORTS`; `policy_for_clone_checkout` and the `CheckoutPolicy` newtype; `policy_for_clone` left carrying all of #188, with the doc comment recording why the first attempt to narrow it was a regression.
- `crates/git-vista-server/src/sandbox/test_env.rs` — new: the crate's one test-side environment writer.
- `crates/git-vista-server/src/sandbox/argv.rs` — `with_ssh_auth_sock` retargeted onto `test_env`; `policy_for_clone_carries_both_188_grants` replaced by its inversion.
- `crates/git-vista-server/src/handlers/clone.rs` — the widened spawn proof; the two `.map(redact_output)` call sites removed as now-carried by the type.
- `docs/adr/0137-*.md` and its rendered PDF in `docs/adr/pdf/`.

Signed: max · 2026-09-07T18:05:00-04:00